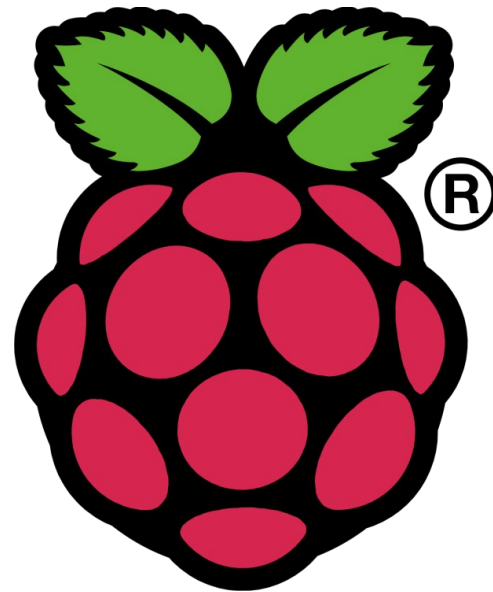


Présentation du Raspberry PI 4



Présentation du Raspberry Pi 4

Créé par la Fondation Raspberry Pi

Apprendre aux enfants à coder

Adopté par les “geeks” et les “makers”

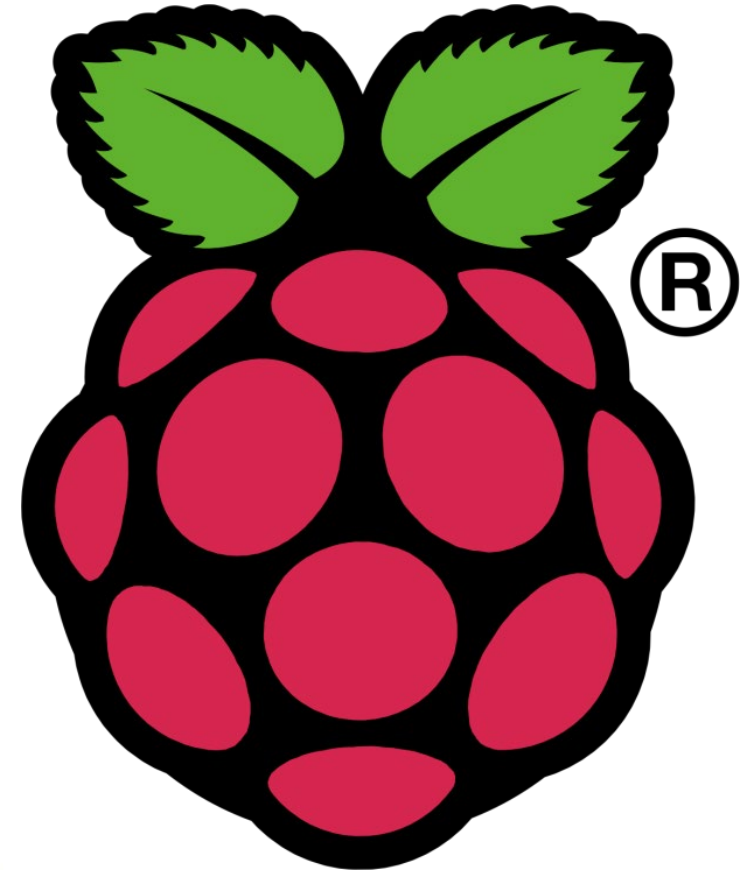
Système d'exploitation libre

Vidéo de qualité

10 000 exemplaires prévus

Plus de **45 000 000** vendus (déc. 2022)!

30% utilisés par les entreprises



Présentation du Raspberry Pi 4

Processeur ARM Cortex-A72

4 Cœurs - 64 bits à 1,5GHz

GPU Broadcom VideoCore VI

2 Sorties HDMI 4K

Mémoire RAM 1, 2, 4 ou 8 Go

Disque dur = Carte SD

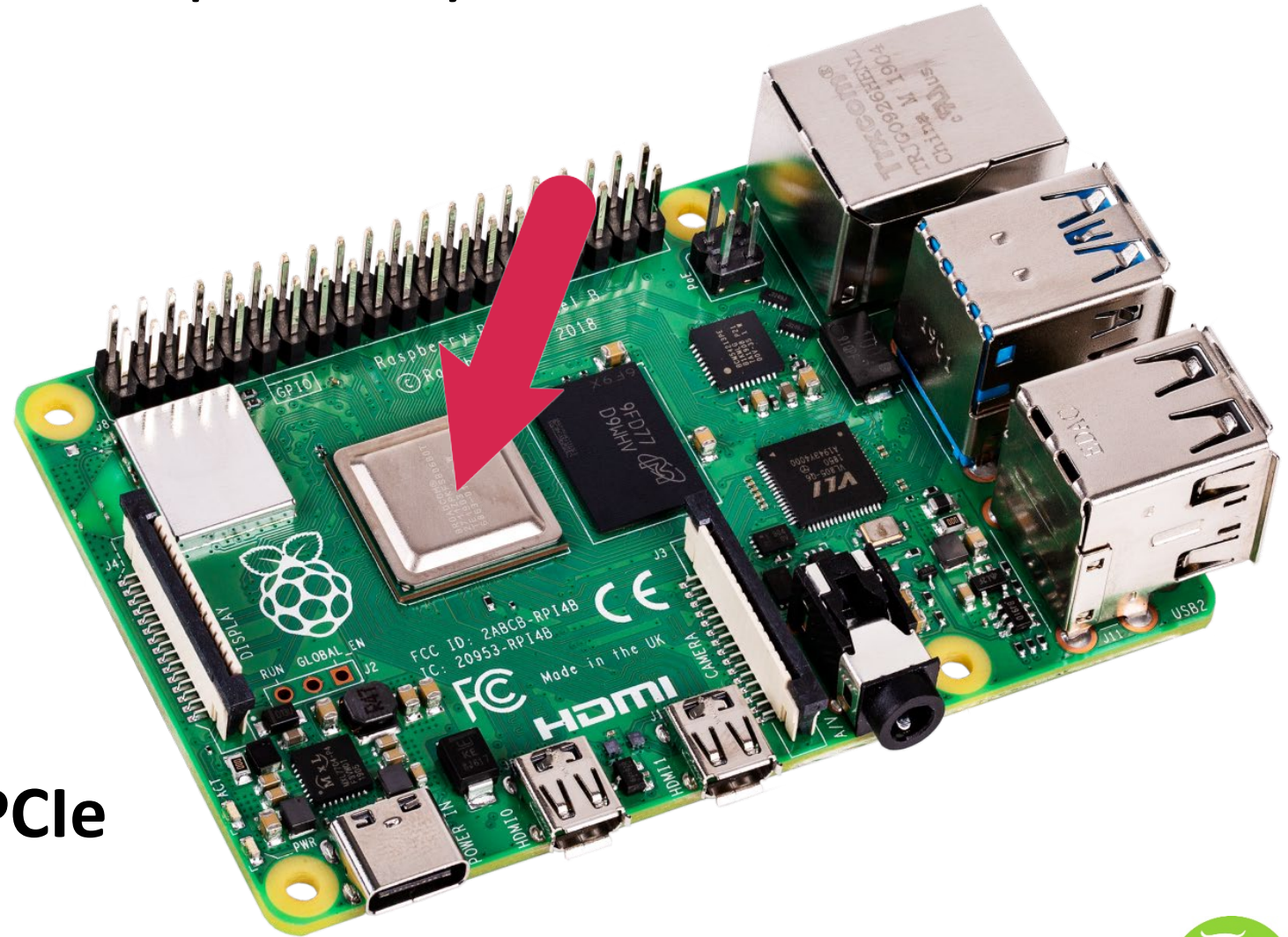
Audio/vidéo analogique

2 ports USB 3.0

2 ports USB 2.0

1 port Ethernet Gigabit

Bus PCIe



Présentation du Raspberry Pi 4

Port Caméra – CSI

Port afficheur – DSI

WiFi 802.11ac et Bluetooth 5.0

40 broches d'entrée/sortie

Connecteur **PoE**

Alimentation **USB-C** 5V/3A

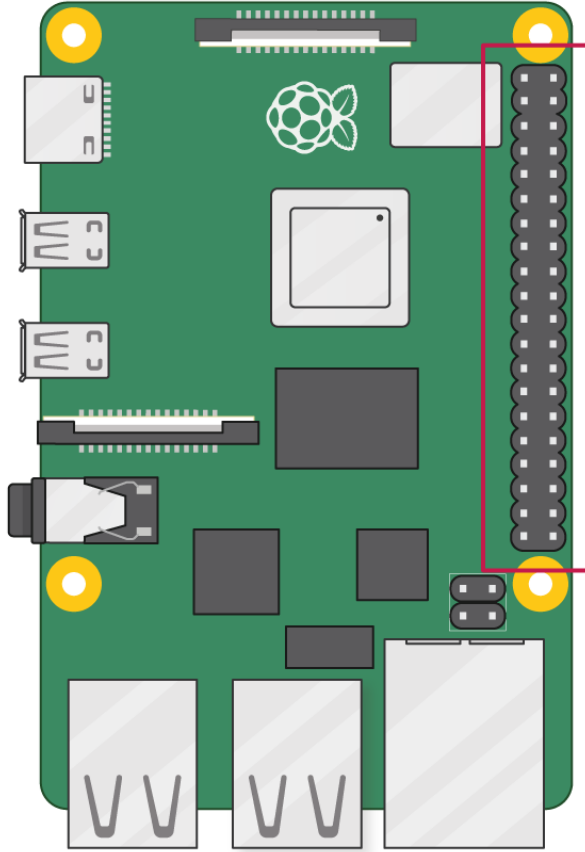
Format carte de crédit (8,5 x 5,6 cm)

45 grammes

48 € - 96€



PINOUT



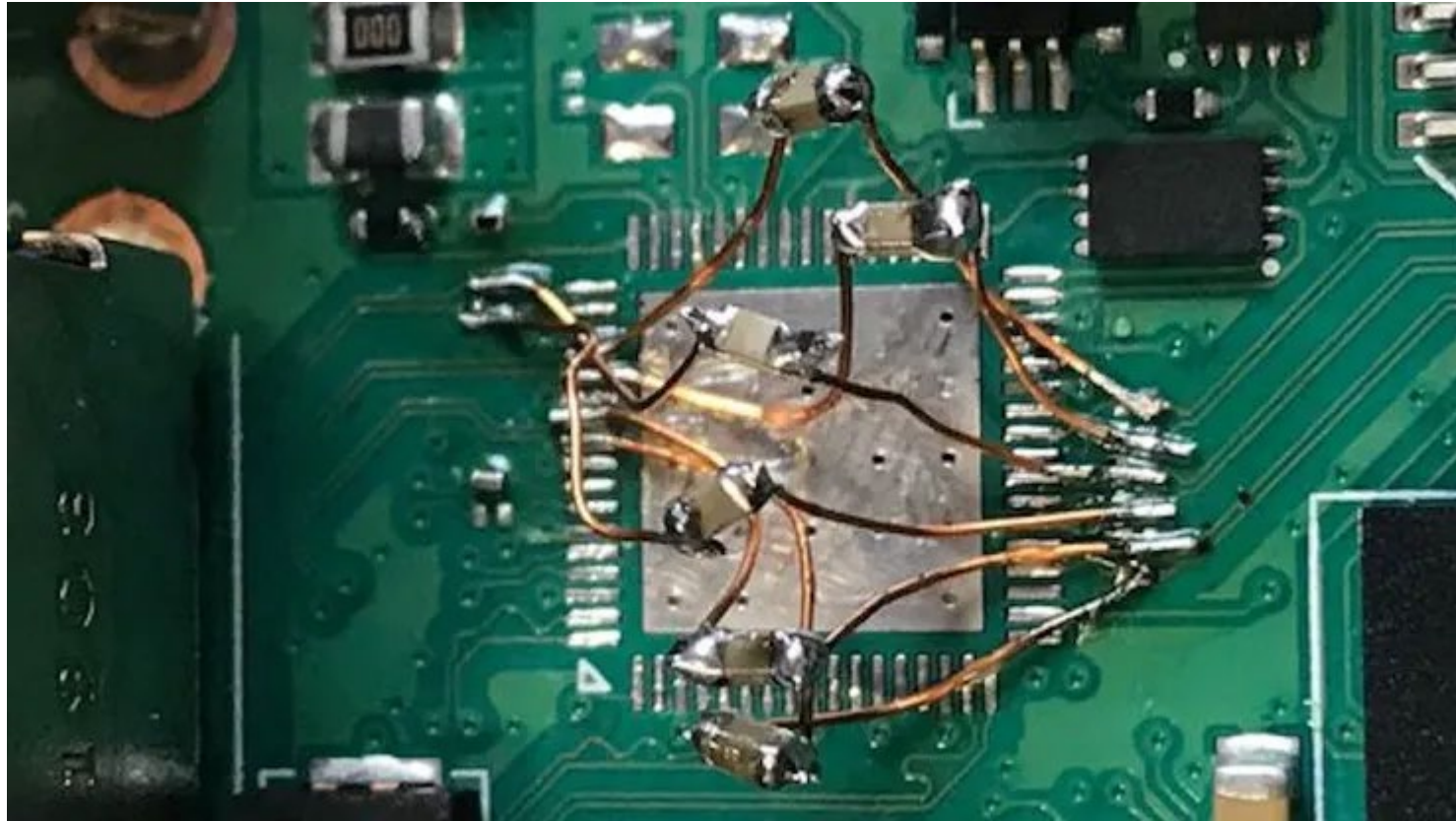
3V3 power	1	2	5V power
GPIO 2 (SDA)	3	4	5V power
GPIO 3 (SCL)	5	6	Ground
GPIO 4 (GPCLK0)	7	8	GPIO 14 (TXD)
Ground	9	10	GPIO 15 (RXD)
GPIO 17	11	12	GPIO 18 (PCM_CLK)
GPIO 27	13	14	Ground
GPIO 22	15	16	GPIO 23
3V3 power	17	18	GPIO 24
GPIO 10 (MOSI)	19	20	Ground
GPIO 9 (MISO)	21	22	GPIO 25
GPIO 11 (SCLK)	23	24	GPIO 8 (CE0)
Ground	25	26	GPIO 7 (CE1)
GPIO 0 (ID_SD)	27	28	GPIO 1 (ID_SC)
GPIO 5	29	30	Ground
GPIO 6	31	32	GPIO 12 (PWM0)
GPIO 13 (PWM1)	33	34	Ground
GPIO 19 (PCM_FS)	35	36	GPIO 16
GPIO 26	37	38	GPIO 20 (PCM_DIN)
Ground	39	40	GPIO 21 (PCM_DOUT)



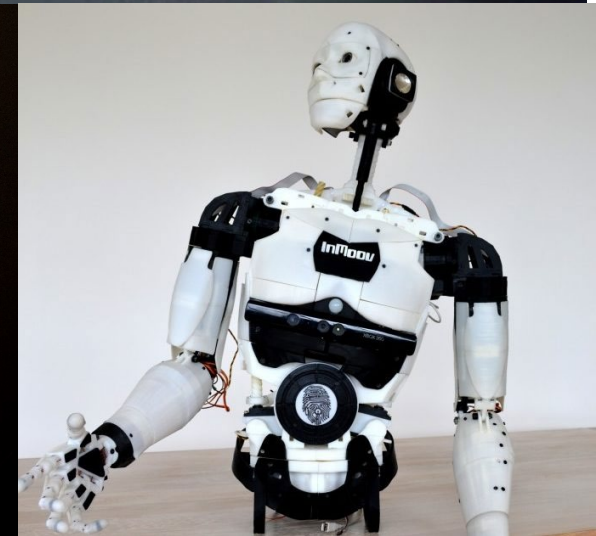
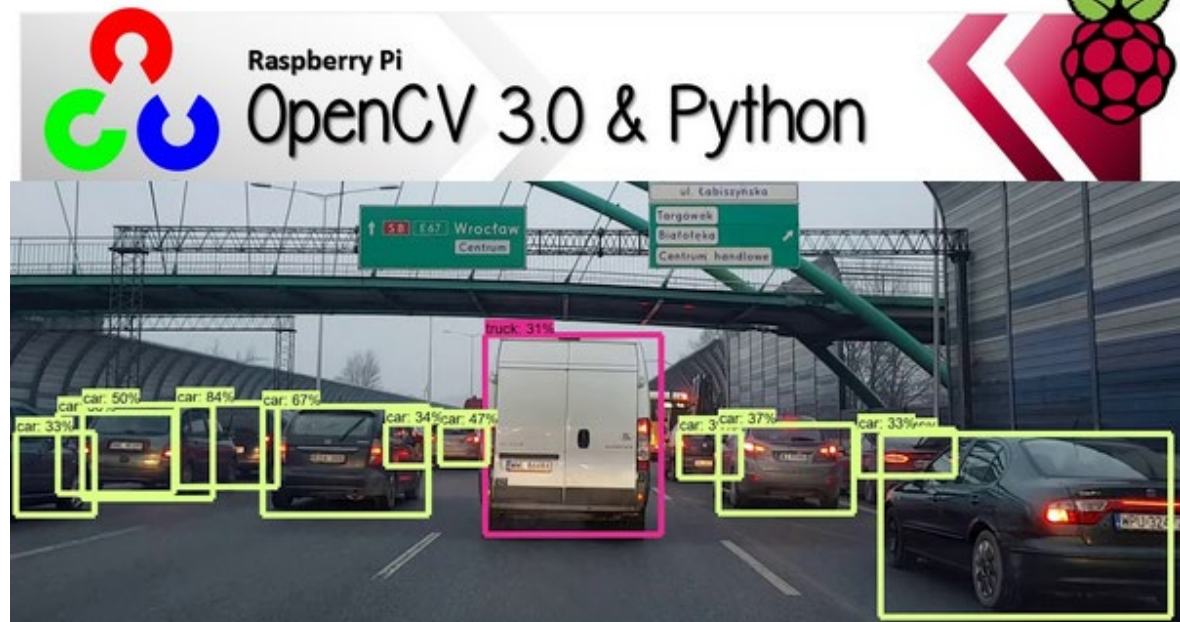
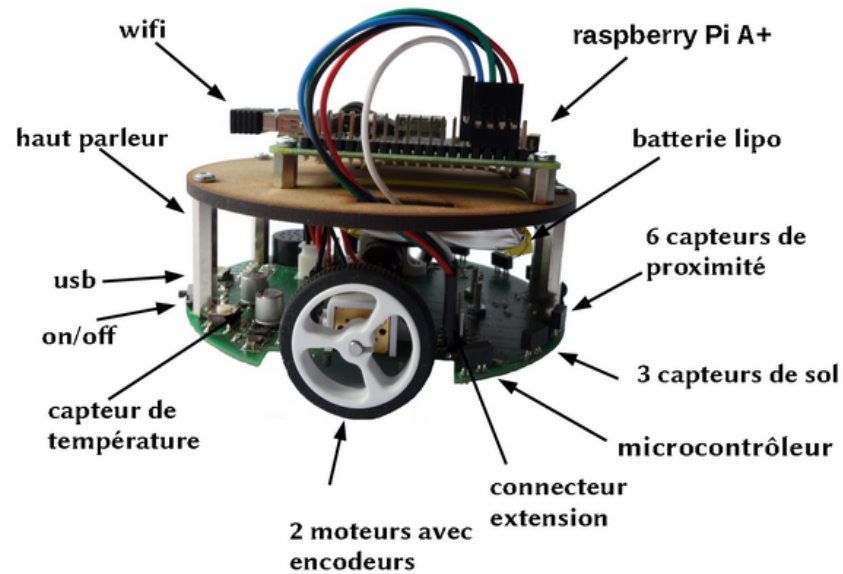
Quelques modules utilisables avec les GPIO



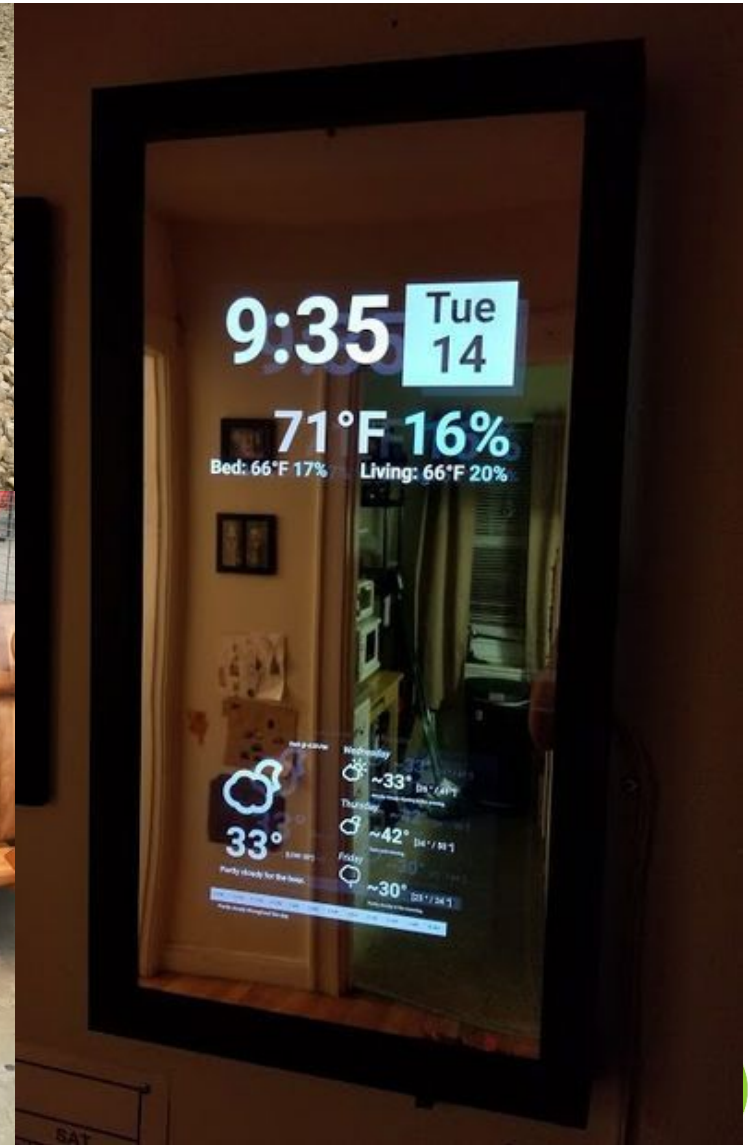
PCIe homemade



Quelques projets



Quelques projets



Installation

Aller sur le site <https://www.raspberrypi.com/software/>

Installer **Raspberry Pi Imager** :

Système d'exploitation

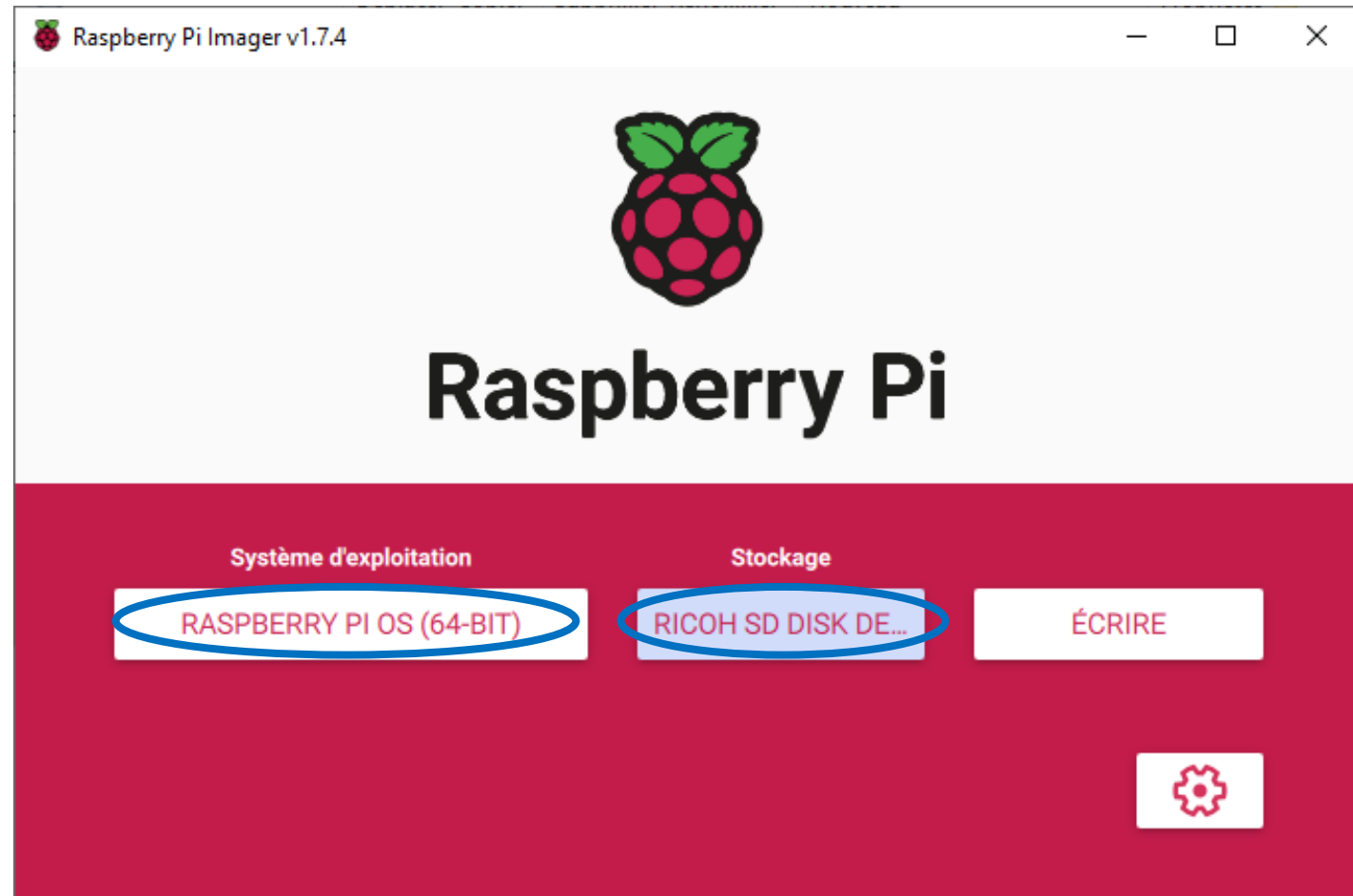
Menu Raspberry Pi OS (other)

Choisir **Raspberry Pi OS (64 bits)**

Stockage

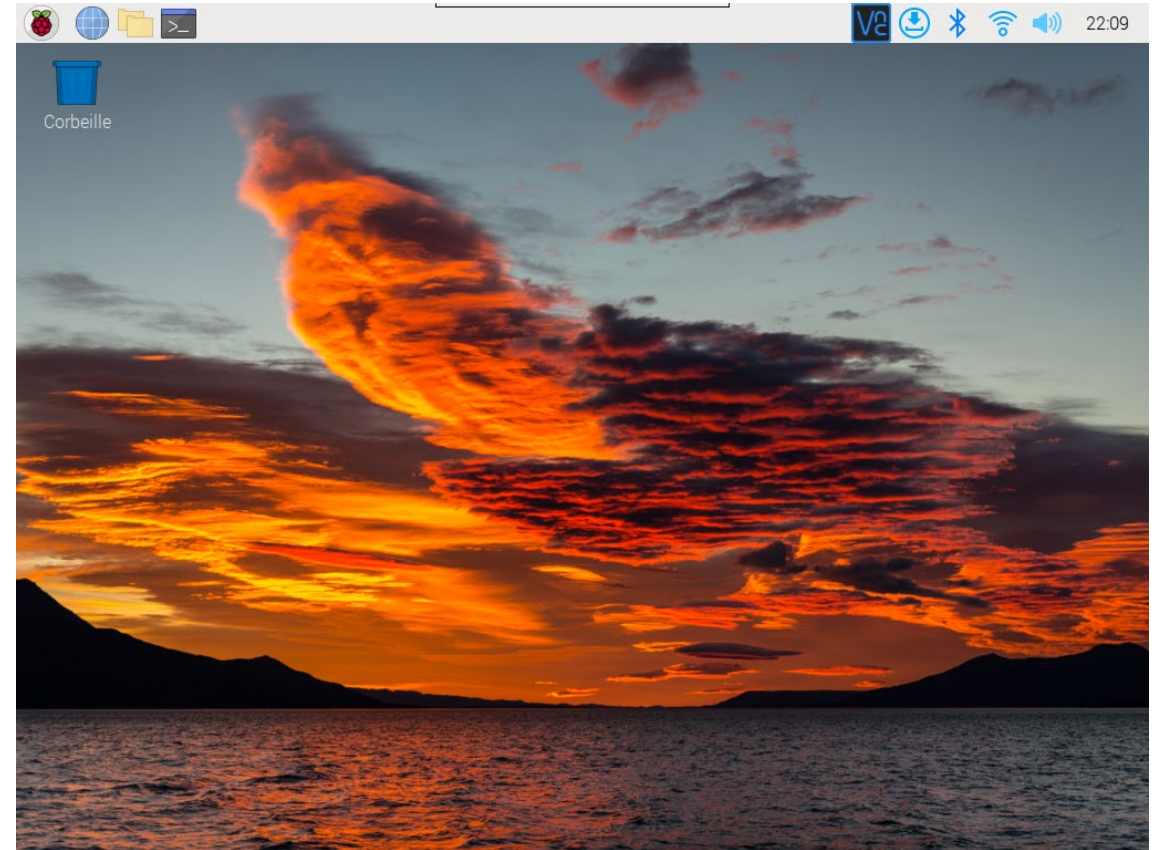
Sélectionner la carte SD

Puis **Ecrire**

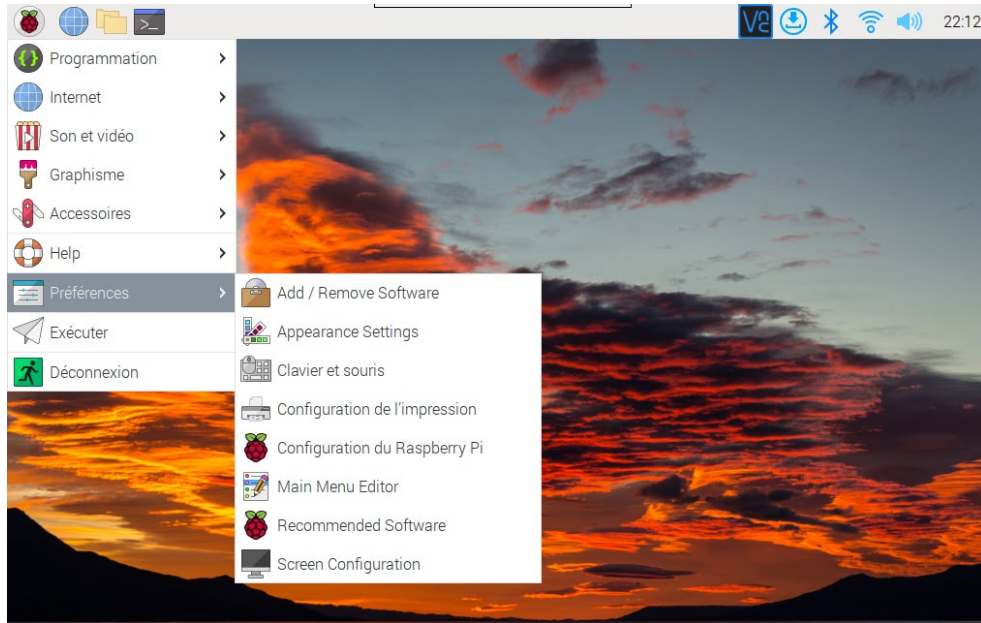


Installation

1. Insérer la carte microSD dans le raspberry Pi
2. Brancher clavier et souris USB
3. Brancher le câble HDMI entre l'écran et le Pi (1^{ère} prise à proximité de l'USB-C
4. Brancher l'alimentation sur l'USB-C
5. Procéder aux étapes d'installation :
 - Sélectionner **France/Français** pour la langue
 - Sélectionner le Wifi et entrer le mot de passe
 - Choisir le nom d'utilisateur **pi** et le mot de passe **raspberry**
 - **Skipper l'étape de mise à jour**



Installation



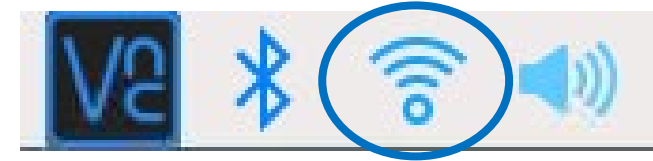
Depuis la framboise (en haut à gauche), menu **Propriétés/Configuration du Raspberry Pi**

Dans l'onglet interface, activer **SSH** et **VNC** puis valider !



Installation

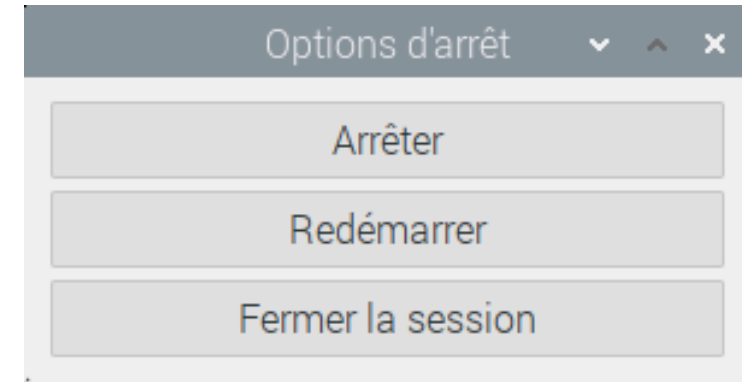
Survoler avec la souris l'icône Wifi en haut à droite :



Noter l'adresse IP, **wlan0**, ex : **192.168.0.33**

A partir de l'icône framboise, utiliser le menu **Déconnexion** et l'option d'arrêt **Arrêter**

Attendre que la led du raspberry soit rouge et stable, puis débrancher, le **clavier**, la **souris**, le **câble HDMI** et enfin le **câble USB-C**



=> Le Raspberry Pi est prêt !

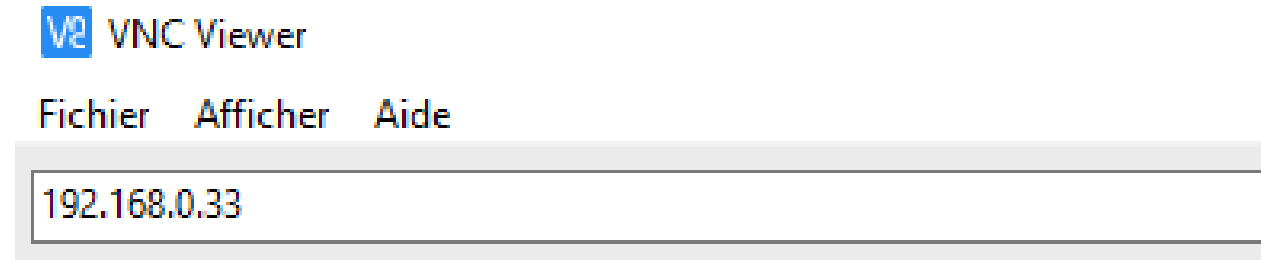


VNC Viewer

Aller sur le lien et télécharger le viewer VNC : <https://www.realvnc.com/en/connect/download/viewer/>

Installer puis Ouvrir VNC Viewer, taper dans l'adresse IP du Raspberry :

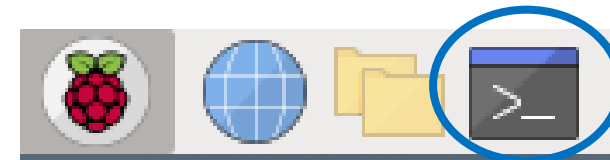
NB : Brancher le câble USB-C du Raspberry en amont !



Utiliser le login et mot de passe définis précédemment, une fenêtre du bureau du Raspberry s'affiche après validation d'une autorisation de connexion !

Ouvrir un terminal et lancer les commandes suivantes :

```
pi@raspberrypi:~ $ sudo apt update
...
pi@raspberrypi:~ $ sudo apt full-upgrade
```



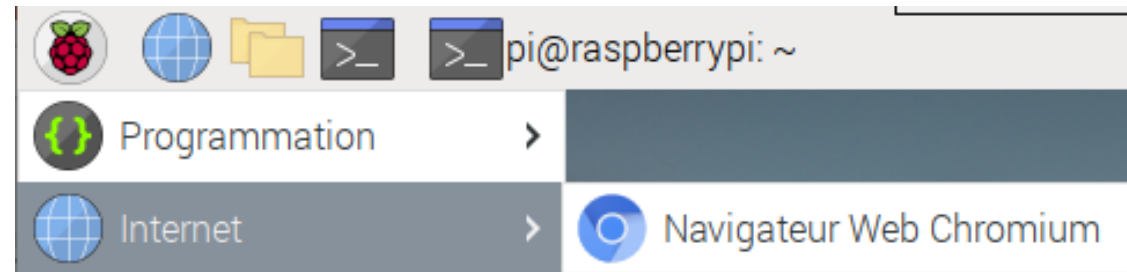
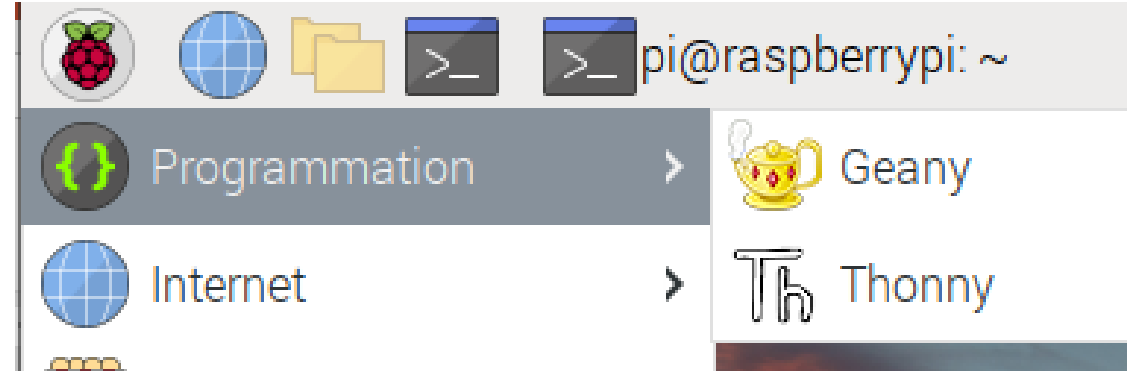
Redémarrer le Raspberry !



Découverte



```
~ $ python --version  
... version de python ...
```



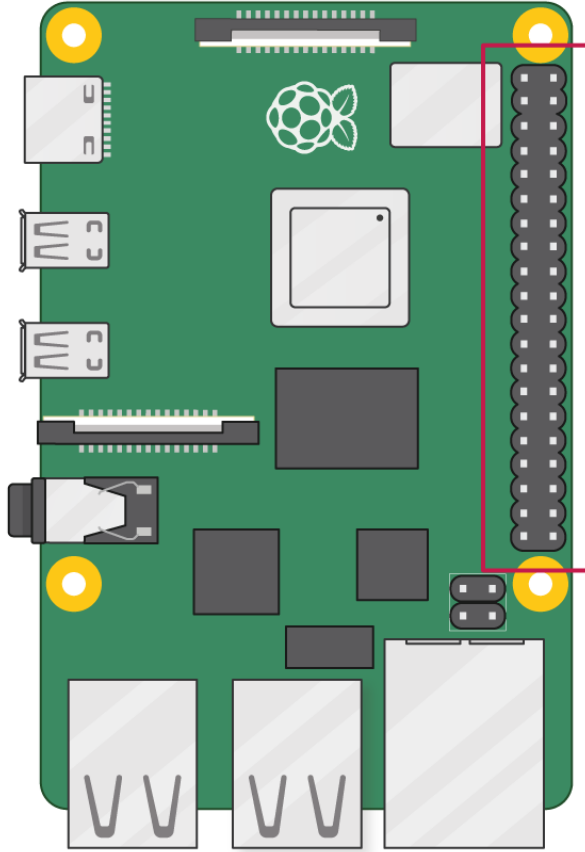
Pour se connecter en SSH :

```
$ ssh pi@pi 123.local
```

```
$ ssh pi@pi 123
```



PINOUT



3V3 power	1	2	5V power
GPIO 2 (SDA)	3	4	5V power
GPIO 3 (SCL)	5	6	Ground
GPIO 4 (GPCLK0)	7	8	GPIO 14 (TXD)
Ground	9	10	GPIO 15 (RXD)
GPIO 17	11	12	GPIO 18 (PCM_CLK)
GPIO 27	13	14	Ground
GPIO 22	15	16	GPIO 23
3V3 power	17	18	GPIO 24
GPIO 10 (MOSI)	19	20	Ground
GPIO 9 (MISO)	21	22	GPIO 25
GPIO 11 (SCLK)	23	24	GPIO 8 (CE0)
Ground	25	26	GPIO 7 (CE1)
GPIO 0 (ID_SD)	27	28	GPIO 1 (ID_SC)
GPIO 5	29	30	Ground
GPIO 6	31	32	GPIO 12 (PWM0)
GPIO 13 (PWM1)	33	34	Ground
GPIO 19 (PCM_FS)	35	36	GPIO 16
GPIO 26	37	38	GPIO 20 (PCM_DIN)
Ground	39	40	GPIO 21 (PCM_DOUT)



 MQTT



MQTT : Message Queuing Telemetry Transport

En 1999, Andy Stanford-Clark (IBM) et Arlen Nipper (Arcom) ont créé un protocole minimaliste (faible consommation électrique et faible bande passante afin de pouvoir monitorer des oléoducs par satellite

Cahier des charges :

- Peu de bande passante (93 fois plus rapide vs HTTP)
 - Protocole léger (entête de message 2 Octets)
 - faible consommation (11 fois moins pour envoyer et 170 pour recevoir vs HTTP)
- Gestion d'une qualité de service (QoS) sur la livraison des données
- Indépendance du format et des données
- Gestion de sessions



MQTT : Architecture Pub/Sub

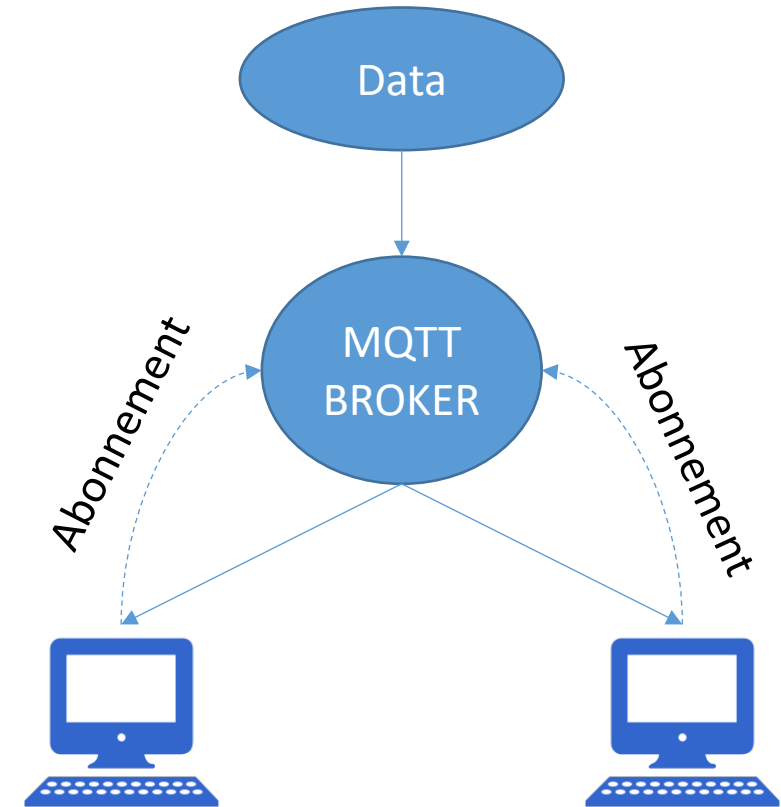
Le BROKER MQTT est le point central

Les clients se connectent au BROKER

Les clients souscrivent à des "TOPICS" (sujets), exemple
`client.subscribe('temperature')`
`client.subscribe('interrupteur')`

Les clients peuvent publier des "PAYLOADS" (messages) sur des "TOPICS", exemple
`client.publish('temperature','24')`
`client.publish('interrupteur', 'ON');`

Tous les clients ayant souscrit à un sujet recevront les messages envoyés pour celui-ci



MQTT : QoS

MQTT utilise TCP qui assure la fiabilité des transmissions sur un réseau. La notion de QoS n'est pas liée à un arrêt du serveur ou du client mais à une éventuelle perte de connexion.

Niveau	action	description
QoS 0	Au plus une fois	«Fire and forget»: un message ne sera pas acquitté par le destinataire ou stocké et renvoyé par l'expéditeur. Niveau par défaut, il correspond à un capteur envoyant des données qui en cas de perte ne perturbe pas le fonctionnement (ex: capteur de température)
QoS 1	Au moins une fois	Un message sera remis au moins une fois au destinataire. Les messages sont assurés d'arriver mais des doublons peuvent se produire.
QoS 2	Exactement une fois	Les messages sont assurés d'arriver exactement une fois. Utilisez ce niveau lorsque des messages dupliqués ou perdus peuvent entraîner des conditions incorrectes.

Un TOPIC publié avec le mode "retained" permet de conserver un « dernier état ». S'utilise pour ne pas perturber un traitement, exemple un interrupteur peut être sur ON ou OFF et il peut être intéressant de connaître sa dernière position connue quand un client se met en écoute.



MOSQUITTO: installation

Ouvrir un terminal et lancer la commande :

```
~ $ sudo apt install -y mosquitto mosquitto-clients
```

Pour démarrer Mosquitto au démarrage du Pi :

```
~ $ sudo systemctl enable mosquitto.service
```

Vérification de la version :

```
~ $ mosquitto -v
```

Pour autoriser une connexion externe :

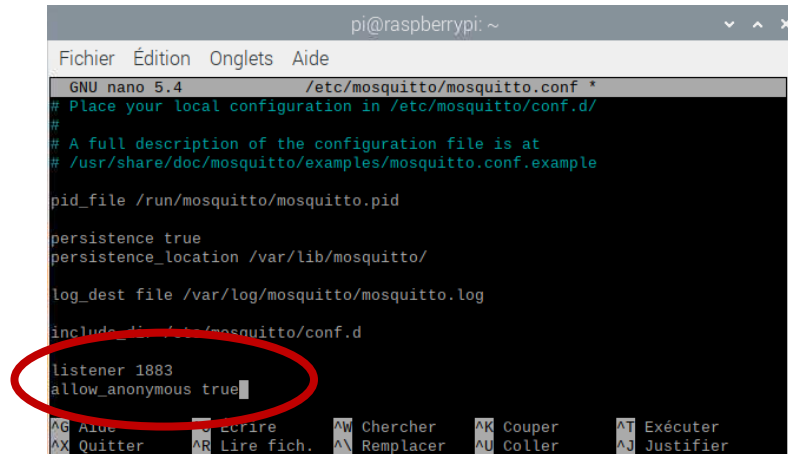
Il faut éditer le fichier de configuration et ajouter deux lignes en fin de fichier :

listener 1883

allow_anonymous true

Naviguer avec les flèches puis valider avec CTRL+X puis O en Entrer

```
~ $ sudo nano /etc/mosquitto/mosquitto.conf
```



```
pi@raspberrypi: ~  
Fichier  Édition  Onglets  Aide  
GNU nano 5.4 /etc/mosquitto/mosquitto.conf *  
# Place your local configuration in /etc/mosquitto/conf.d/  
#  
# A full description of the configuration file is at  
# /usr/share/doc/mosquitto/examples/mosquitto.conf.example  
  
pid_file /run/mosquitto/mosquitto.pid  
  
persistence true  
persistence_location /var/lib/mosquitto/  
  
log_dest file /var/log/mosquitto/mosquitto.log  
  
include_dir /etc/mosquitto/conf.d  
  
listener 1883  
allow_anonymous true  
AG Aide      AR Écrire    AN Chercher  AK Couper    AT Exécuter  
AX Quitter   AL Lire fich.  AM Remplacer AU Coller    AJ Justifier
```

```
~ $ sudo systemctl restart mosquitto
```

Redémarrer le serveur :



MOSQUITTO: Exemple

Ouvrir un premier terminal et lancer la commande :

```
~ $ mosquitto_sub -h localhost -t temperature
```

Ouvrir un second terminal et lancer la commande :

```
~ $ mosquitto_pub -h localhost -t temperature -m 22
```

Le premier terminal doit afficher :

```
~ $ mosquitto_sub -h localhost -t temperature  
22
```

Demander à un autre stagiaire, l'IP de son raspberry et lancer la commande dans le second terminal :

```
~ $ mosquitto_pub -h 192.168.0.34 -t temperature -m 23
```

Le premier terminal du 192.168.0.34 doit afficher le message !!!



MOSQUITTO et python

Installer la librairie Paho-MQTT via un terminal :

```
~ $ pip install paho-mqtt
```

Ouvrir **Thonny** et lancer le script suivant :

Toutes les 30 secondes vous recevrez la température et l'humidité de la pièce !

```
import paho.mqtt.client as mqtt

mqtt_broker = "mesures.ludiksciences.fr"
mqtt_port = 1883
mqtt_topic = "DTH22"

def on_message(client, userdata, message):
    print(message.payload.decode())

client = mqtt.Client()
client.on_message = on_message
client.connect(mqtt_broker, mqtt_port)

client.subscribe(mqtt_topic)

client.loop_forever()
```



MOSQUITTO et python

Ouvrir **Thonny** et lancer le script suivant :

Ce script permet d'envoyer un message sur le topic
Python

Mettez vous en écoute soit via le terminal soit via le script précédent en modifiant le **topic** !!!

```
import paho.mqtt.client as mqtt

mqtt_broker = "mesures.ludiksciences.fr"
mqtt_port = 1883
mqtt_topic = "Python"

def on_publish(client, userdata, message):
    print(mqtt_payload)

client = mqtt.Client()
client.on_publish = on_publish
client.connect(mqtt_broker, mqtt_port)
mqtt_payload = "Georges"
client.publish(mqtt_topic, mqtt_payload)
```



Pico et MQTT

Via Thonny et le menu Outils/Gérer les paquets, installer umqtt.simple

The image displays two screenshots of the Thonny package manager interface for Raspberry Pi Pico @ COM3.

Left Screenshot: The search results for `umqtt.simple` are shown. The search bar contains `umqtt.simple`. The button `Rechercher sur PyPI` is visible. The results list includes:

- `micropython-umqtt.simple` (highlighted): Lightweight MQTT client for MicroPython.
- `pycopy-umqtt.simple`: Lightweight MQTT client for Pycopy.
- `micropython-umqtt.simple2`: MQTT client for MicroPython.
- `micropython-umqtt.robust`: Lightweight MQTT client for MicroPython ("robust" version).
- `micropython-umqtt.robust2`: MQTT client for MicroPython ("robust" version).
- `pycopy-umqtt.robust`: Lightweight MQTT client for Pycopy ("robust" version).
- `simple503`

Right Screenshot: The search results for `umqtt.simple` are shown. The error message is displayed: "Impossible de trouver le paquet dans PyPI Code d'erreur : <urlopen error [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: certificate has expired (_ssl.c:997)>". The `Installer` button is visible.



Pico et MQTT

```
import network
import time
from machine import Pin
from umqtt.simple import MQTTClient

SSID = "BJ16"
PASSWORD = "BJ16@2023"
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(SSID,PASSWORD)
if not wlan.isconnected():
    wlan.connect(SSID,PASSWORD)
    print("Waiting for connection...")
    while not wlan.isconnected():
        time.sleep(1)
mqtt_server = '192.168.0.33'
mqtt_topic = 'Python'
```

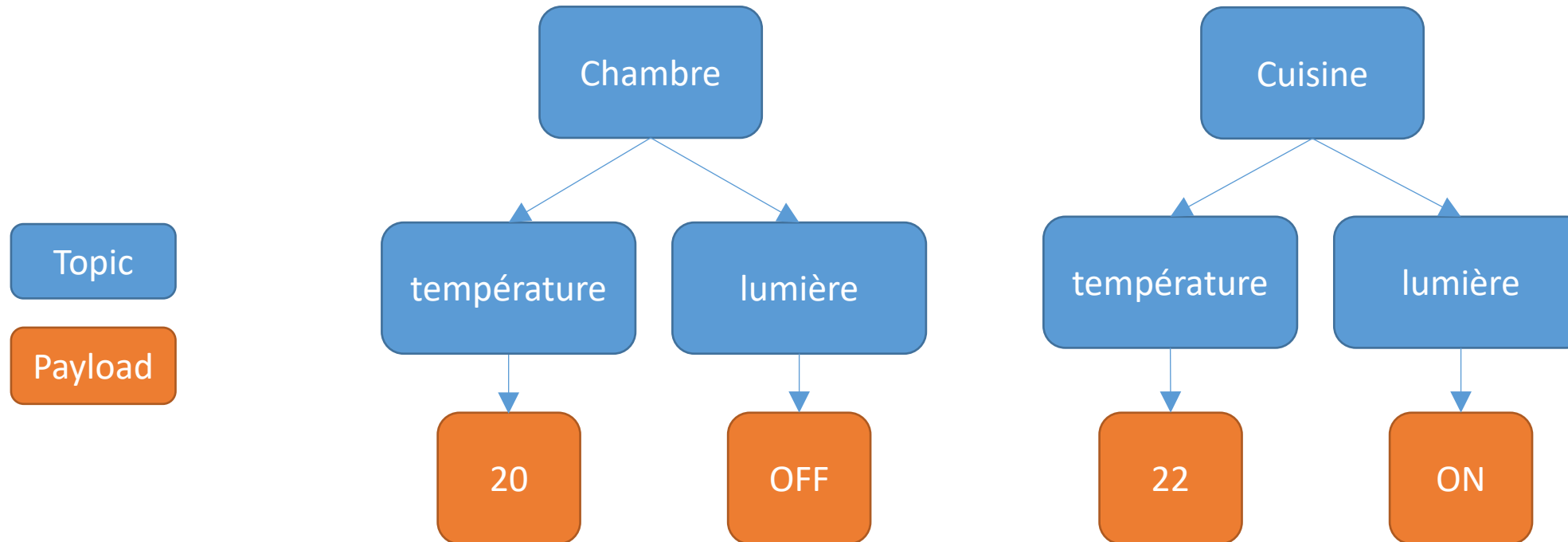
```
def reconnect():
    print('Failed to connect to MQTT')
    machine.reset()
try :
    client = MQTTClient("moi", mqtt_server)
    client.connect()
except OSError as e:
    reconnect()
while True:
    for i in range(10):
        client.publish(mqtt_topic, str(i))
        time.sleep(0.5)
```

Lancer le script sur le Pico et utilisez le premier script MQTT sur le Raspberry pour visualiser les données

NB : Modifiez le topic !!!



MQTT : Hierarchie des Topics



Chambre/température : 20

Chambre/lumière : OFF

Cuisine/température : 22

Cuisine/lumière : ON

+ permet d'accéder aux TOPICS d'un même niveau

permet d'accéder à tous les TOPICS sous une hiérarchie

- 1) Comment souscrire à toutes les températures
- 2) A tous les capteurs de la cuisine



SERVEUR WEB/APACHE

Apache représente 44 % de tous les serveurs web dans le monde

Installer Apache :

```
~ $ sudo apt install apache2 -y
```

Les fichiers se trouvent dans le répertoire
/var/www/html

Modifier les droits pour pouvoir écrire :

```
~ $ sudo usermod -a -G www-data pi  
~ $ sudo chown -R -f www-data:www-data /var/www/html  
~ $ sudo chmod 755 /var/www
```

Redémarrer le Raspberry pour prendre en compte
les modifications !



CRÉER UNE PAGE WEB

Depuis le terminal :

Supprime le fichier index.html

```
~ $ sudo rm /var/www/html/index.html
```

Créer un fichier index.html

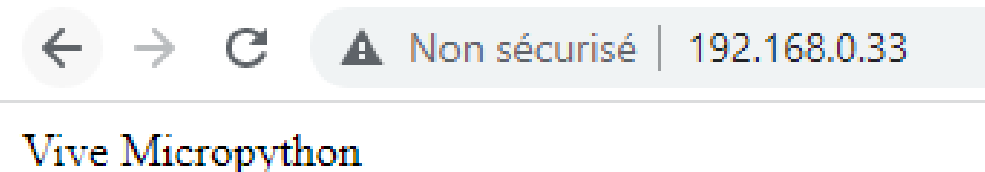
```
~ $ sudo nano /var/www/html/index.html
```

Puis insérer le code suivant :

Puis CTRL-X, O

```
<html>
<head></head>
<body>
Vive Micropython
</body>
</html>
```

Ouvrir un navigateur sur votre machine et insérer l'adresse de votre Raspberry dans la barre d'adresse



Python et Apache

Depuis le terminal

```
~ $ sudo apt-get install libapache2-mod-wsgi-py3
```

Créer un dossier pour notre application

```
~ $ mkdir /home/pi/iot
```

Créer le fichier WSGI (Web Server Gateway Interface)

```
~ $ nano /home/pi/iot/iot.wsgi
```

Insérer dans le fichier

```
import sys
sys.path.insert(0, '/home/pi/iot')
from iot import app as application
```



Python et Apache

Créer le fichier de configuration

```
~ $ sudo nano /etc/apache2/sites-available/iot.conf
```

Insérer

```
<VirtualHost *:80>
    ServerName iot
    WSGIDaemonProcess iot user=pi group=www-data threads=5
    WSGIScriptAlias /iot /home/pi/iot/iot.wsgi
    <Directory /home/pi/iot>
        WSGIProcessGroup iot
        WSGIScriptReloading On
        WSGIApplicationGroup %{GLOBAL}
        Require all granted

    </Directory>
    ErrorLog /home/pi/iot/logs/error.log

</VirtualHost>
```



Python et Apache

Créer le répertoire de logs

```
~ $ mkdir /home/pi/iot/logs
```

Activer WSGI

```
~ $ sudo a2enmod wsgi
```

Activer la configuration WSGI

```
~ $ sudo /usr/sbin/a2ensite iot.conf
```

Désactiver la configuration par défaut

```
~ $ sudo /usr/sbin/a2dissite 000-default.conf
```

Redémarrer Apache

```
~ $ sudo systemctl reload apache2
```

Dans /home/pi/iot/ créer un fichier iot.py contenant

```
from flask import Flask

app = Flask(__name__)

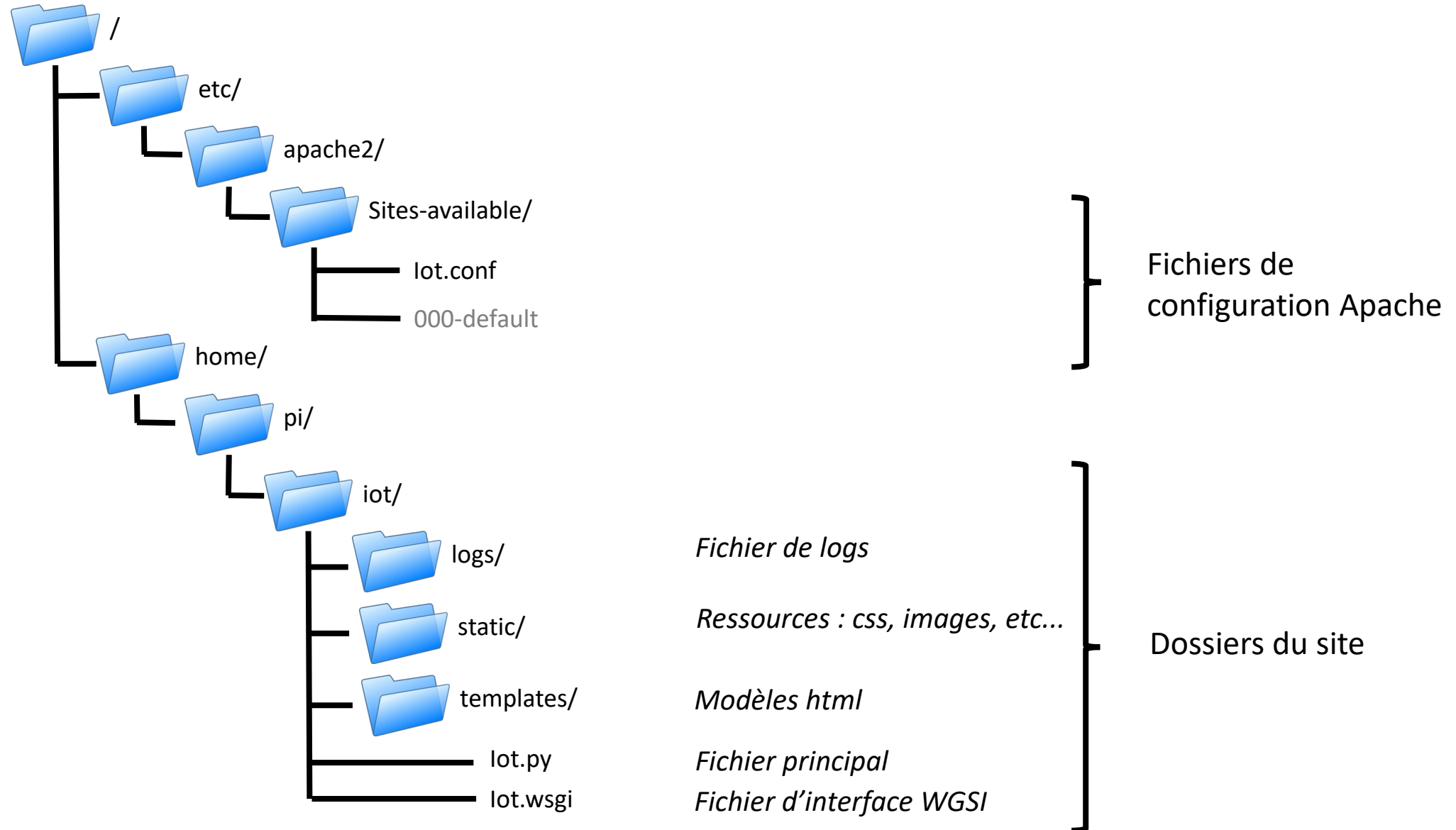
@app.route("/")
def hello():
    return "Bonjour à tous !"

if __name__ == "__main__":
    app.run(debug = True)
```

Dans un navigateur, ouvrir l'url :
192.168.0.33/iot



Structure d'un site Flask



Utilisation d'un template

Créer un répertoire **templates** dans
/home/pi/iot

Dans le répertoire /home/pi/iot/templates,
créer un fichier bonjour.html et insérer :

```
<!DOCTYPE html >
<html>
  <head>
    <title>Mon site Web</title>
  </head>
  <body>
    <h1>Bonjour {{name }} !</h1>
    <p>Nous sommes le {{ date }}</p>

    <dl>
      {% for cle, valeur in dico.items() %}
        <dt>{{ cle }} :</dt>
        <dd>{{ valeur }}</dd>
      {% endfor %}
    </dl>

  </body>
</html>
```



Utilisation d'un template

Dans le répertoire /home/pi/iot/, ajouter au fichier iot.py :

```
from flask import Flask, render_template

def hello():
    return "Bonjour à tous !"

from datetime import datetime
@app.route("/bonjour")
def bonjour():
    today = datetime.now()
    date = today.strftime("%d:%m:%Y %H:%M:%S")
    name = "Georges"
    dico = {'Python' : '70', 'IoT' : '100'}
    return render_template("bonjour.html", date = date, name = name, dico = dico)

if __name__ == "__main__":
    app.run(debug = True)
```

Dans un navigateur, ouvrir l'adresse : <http://192.168.0.33/iot/bonjour>



Base de données : Création

Dans le répertoire `/home/pi/iot/`, créer un fichier **schema.sql**:

```
DROP TABLE IF EXISTS capteur;

CREATE TABLE capteur (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data TEXT NOT NULL
);
```

Dans le répertoire `/home/pi/iot/`, créer un fichier **init_db.py** :

```
import sqlite3

connection = sqlite3.connect('/home/pi/iot/database.db')

with open('/home/pi/iot/schema.sql') as f:
    connection.executescript(f.read())

connection.commit()
connection.close()
```

Exécuter le fichier via le terminal pour créer la base :

```
~ $ python /home/pi/iot/init_db.py
```



Base de données : Insertion

Nous allons créer une adresse permettant d'insérer des données dans la base, en modifiant **iot.py**

... Ancien code ...

```
import sqlite3

def get_db_connection():
    conn = sqlite3.connect('/home/pi/iot/database.db')
    conn.row_factory = sqlite3.Row
    return conn

@app.route('/data/<userdata>')
def insert_data(userdata):
    conn = get_db_connection()
    post = conn.execute(f'INSERT INTO capteur (data) VALUES ({userdata})')
    conn.commit()
    conn.close()
    return f'Donnée insérée :{userdata}'

if __name__ == "__main__":
    app.run(debug = True)
```

Dans un navigateur, ouvrir l'adresse : <http://192.168.0.33/iot/data/22>



Base de données : Afficher les datas

Dans le répertoire /home/pi/iot/templates, créer un fichier **data.html** et insérer :

```
<!DOCTYPE html >
<html>
  <head>
    <title>Les datas</title>
  </head>
  <body>
    {% for data in datas %}
      <div class='data'>
        <p>{{ data['created'] }} => {{ data['data'] }}</p>
      </div>
    {% endfor %}

  </body>
</html>
```



Base de données : Afficher les datas

Nous allons créer une adresse permettant d'afficher les données de la base, en modifiant iot.py

... Ancien code ...

```
@app.route('/data')
def mes_datas():
    conn = get_db_connection()
    datas = conn.execute('SELECT * FROM capteur').fetchall()
    conn.close()
    return render_template('data.html', datas=datas)

if __name__ == "__main__":
    app.run(debug = True)
```

Dans un navigateur, ouvrir l'adresse : <http://192.168.0.33/iot/data>



Base de données : Afficher un graphique

Installer matplotlib via le terminal :

```
~ $ pip install matplotlib
```

Modifier **iot.py** :

... Ancien code ...

```
import matplotlib.pyplot as plt
import io
import base64

@app.route('/plot')
def build_plot():
    conn = get_db_connection()
    datas = conn.execute('SELECT * FROM capteur').fetchall()
    conn.close()
    img = io.BytesIO()
    x=list(range(len(datas)))
    y=[]
    for data in datas :
        y.append(int(data['data']))
    plt.plot(x,y)
    plt.savefig(img, format='png')
    img.seek(0)
    plot_url = base64.b64encode(img.getvalue()).decode()
    return ''.format(plot_url)

if __name__ == "__main__":
    app.run(debug = True)
```



Installation

Procéder aux étapes d'installation, seule l'étape de mise à jour finale peut être saignée

```
html = """<!DOCTYPE html><html>  
<head></head>  
<body>Bonjour</body></html>"""
```

```
cl.send('HTTP/1.0 200 OK\r\n/  
Content-type: text/html\r\n\r\n')  
  
cl.send(response)  
cl.close()
```

Fichier webserveur.py



LUDIKSCIENCES